



University of Camerino

SCHOOL OF SCIENCES AND TECHNOLOGIES

Master degree in Computer Science (Class LM-18)

Homework Exam 06/03/2026

Fundamentals of Machine Learning

Francesco Caldarelli - 132380

A.A. 2025/2026

Contents

1	Introduction	5
2	Exercises	7
2.1	Exercise 1 regression1.csv	7
2.2	Exercise 2 classification1.csv	8
2.3	Exercise 3 regression2.csv	9
2.4	Exercise 4 classification2.csv	9
3	Conclusions	11

1. Introduction

Building a machine learning pipeline is mostly about looking at the data first and finding a model that actually fits it, rather than reaching for the most powerful algorithm available and hoping for the best. In this project I worked on four different datasets, two set up as regression problems and two as classification problems, and for each of them I tried to let the data itself decide which model made sense, instead of picking one upfront.

The process was roughly the same for all four exercises. First I plotted the data and looked at it: is it smooth, is it broken somewhere, are the classes mixed together or clearly apart. Based on that I picked one or more candidate models and tested them properly, using cross-validation instead of trusting a single train/test split. Where a hyperparameter was involved, mainly the neighborhood size k in k -Nearest Neighbors, I did not fix it in advance: I tracked how the cross-validated error changed across a range of values and picked the point where it stopped improving.

What came out of this is that the same algorithm can behave in almost opposite ways depending on the dataset. In one exercise the right amount of smoothing was basically none at all, in another it was a lot, and in a third exercise a much simpler linear model turned out to be enough on its own. The rest of this report goes through each of the four exercises in that order, with the reasoning and the cross-validation results that led to the final choice.

2. Exercises

Code Execution and Reproducibility

To ensure full reproducibility of the results discussed in this report, the accompanying Python scripts can be executed from the terminal.

First, ensure that a Python 3 environment is active. It is recommended to install the required dependencies using:

```
pip install -r requirements.txt
```

Once the environment is configured, each exercise can be run independently. For example, to generate the output for the first exercise, run:

```
python exercise1.py
```

Each script will automatically read the corresponding input dataset and generate a `result.csv` file containing the model's predictions over the specified grid.

2.1 Exercise 1 `regression1.csv`

The first thing that stands out when plotting `regression1.csv` is a sharp break in the data. Around $x = 1.8$ the y values drop to a constant value close to -7×10^{10} , which is about six orders of magnitude larger in absolute value than everything else in the dataset. Below that point, the remaining 300 samples trace a smooth curve with no obvious sign of noise.

A single global model cannot really deal with both of these regimes at once. A linear or polynomial fit over the whole range would either get dragged completely off course by the huge values in the flat region, or it would try to average across the jump and end up describing neither region well. For this reason I went with a k -NN regressor, since it only looks at the closest training points to make a prediction and does not need to assume anything about the shape of the function everywhere.

The only real question left was how many neighbors to use. I ran a 5-fold cross-validation over a range of k from 1 to 15, and the result was not subtle at all: the mean squared error jumps by about 13 orders of magnitude just going from $k = 1$ to $k = 2$. That happens because as soon as k is 2 or more, some of the neighborhoods near the transition point end up containing both a "normal" value and one of the extreme flat values, and averaging the two produces a prediction that is wrong on both sides. With $k = 1$ each prediction comes from a single nearest point, so the model just reproduces the sampled function directly, jump included. I locked k to 1 and used that for the final model, shown in Figure 2.1.

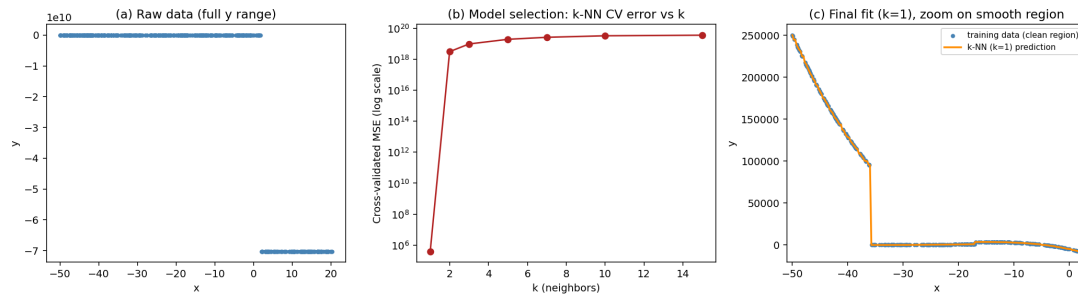


Figure 2.1: (a) Geometric discontinuity in `regression1.csv`. (b) Cross-validated MSE vs k , showing $k = 1$ as the clear minimum. (c) Final k -NN ($k = 1$) fit on the smooth region.

2.2 Exercise 2 `classification1.csv`

Compared to the first exercise, `classification1.csv` is a much simpler case. Plotting the 100 samples by class shows three groups that occupy clearly separate areas of the plane, mostly split along the x_1 axis, with very little overlap between them (Figure 2.2a).

Since the classes looked reasonably well separated, I tested a few candidates side by side with 5-fold stratified cross-validation: multinomial logistic regression, a linear SVC, an RBF-kernel SVC, and a k -NN classifier with $k = 5$. Logistic regression came out slightly ahead with 88.0% mean accuracy, against 86.0% for the linear SVC, 84.0% for the RBF SVC and 85.0% for k -NN (Figure 2.2b). The differences between these numbers are small given only 100 samples and are well within the standard deviation across folds, so this is not a case where one model is dramatically better than the rest.

Given that, I kept logistic regression as the final model rather than one of the more flexible alternatives. It is not meaningfully worse than the others here, it is much cheaper to fit and to run, and it gives a decision boundary that is easy to interpret. It also returns class probabilities directly through `predict_proba`, which I used to plot the confidence contours in Figure 2.2c: probability is close to 1 near the middle of each cluster and drops toward the borders between classes, which is exactly where a wrong prediction is more likely.

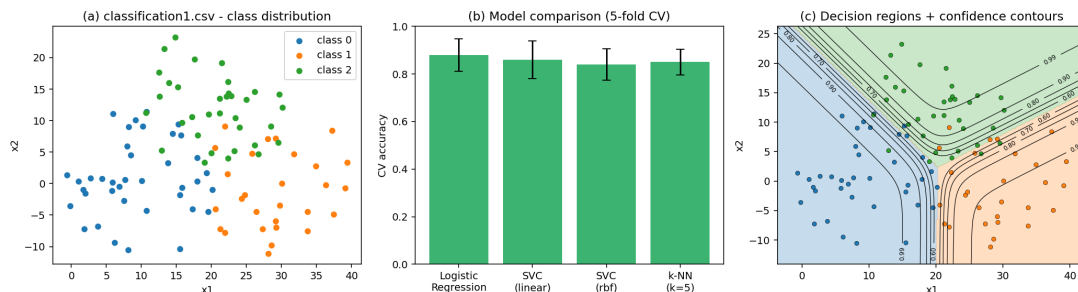


Figure 2.2: (a) Class distribution in `classification1.csv`. (b) Cross-validated accuracy across the four candidate models. (c) Decision regions and confidence contours for the final logistic regression model.

2.3 Exercise 3 regression2.csv

regression2.csv turned out to be the exercise where I spent the most time before landing on something that worked. The dataset is a bivariate surface, 10,000 points with y roughly between 0.04 and 0.84, and unlike Exercise 1 there is no clean break anywhere, just what looks like a smooth trend buried under a fair amount of noise.

I first tried polynomial regression, mostly to have a baseline. It topped out around $R^2 \approx 0.23$ even at degree 5, which told me the surface is not something a low-degree polynomial can describe well, at least not on its own.

Moving to k -NN, my first instinct was to try small-ish values, $k = 5$ and $k = 10$, assuming that would be a reasonable middle ground between following the data closely and smoothing out the noise. The result was not good: R^2 stayed low, not far from what I was getting with the polynomial fit, and the predicted surface still looked noisy when I plotted it. At that point it was clear the issue was the opposite of what I expected: the neighborhoods were too small, not too large, so the model was still fitting local noise instead of averaging it out. I widened the cross-validation search considerably and let k go much higher than I initially thought would make sense.

The result surprised me a bit: cross-validated R^2 kept climbing as k increased, from 0.04 at $k = 3$ up to about 0.26 around $k = 100$, where it flattens out (Figure 2.3b). It's basically the opposite situation from Exercise 1: there, a single neighbor was enough because the data wasn't noisy, just discontinuous. Here the neighborhoods need to be large to average away the noise, since a single point (or even 10) doesn't tell you much about the underlying trend on its own. With $k = 100$ the reconstructed surface (Figure 2.3c) looks much closer to a smooth underlying shape than the raw scatter does.

I should note that an R^2 around 0.26 is still fairly low in absolute terms. This suggests that a large part of the variance in y is genuinely noise that no model, however well tuned, is going to explain away.

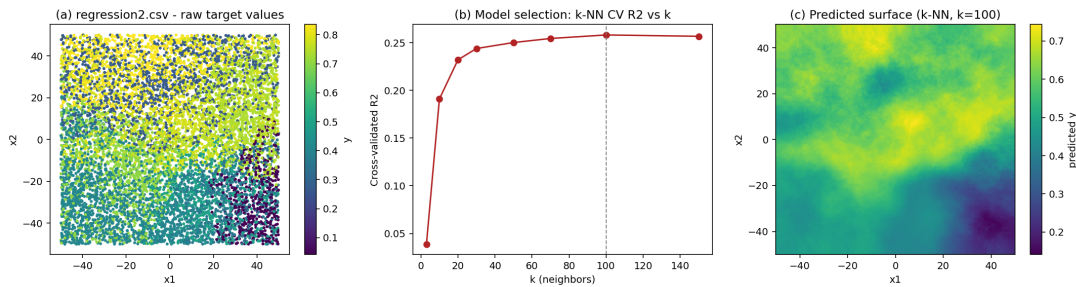


Figure 2.3: (a) Raw noisy surface in regression2.csv. (b) Cross-validated R^2 vs k , plateauing around $k = 100$. (c) Final surface reconstruction with $k = 100$.

2.4 Exercise 4 classification2.csv

The last dataset, classification2.csv, has 10,000 points split between two classes that overlap heavily across most of the input space (Figure 2.4a). As a first check I ran logistic regression as a baseline, mostly to see whether a linear boundary would be anywhere close to sufficient. It reached only 65.6% cross-validated accuracy, which is well above chance for a balanced two-class problem but far from good, confirming the

boundary between the classes is not linear.

Switching to k -NN, I ran cross-validation across a wide range of k values (Figure 2.4b). With $k = 1$ accuracy was 68.5%, only slightly better than the linear baseline, since with a single neighbor the model is essentially memorizing local noise rather than learning a general boundary. As k increases, accuracy climbs fairly quickly at first, reaching about 74.2% at $k = 21$, then continues improving only marginally after that, up to around 74.5–74.8% somewhere between $k = 30$ and $k = 200$, with no clear further gain past that range.

I picked $k = 21$ as the final value. It sits close to where the curve starts to flatten, so pushing k much higher buys very little extra accuracy while making every single prediction more expensive to compute, since it requires averaging over more neighbors. The residual error, sitting around 25% no matter how large k gets, is most likely not something a better choice of k can fix: it looks like the two classes genuinely overlap in this region of feature space, so some fraction of points are simply going to be misclassified regardless of the model.

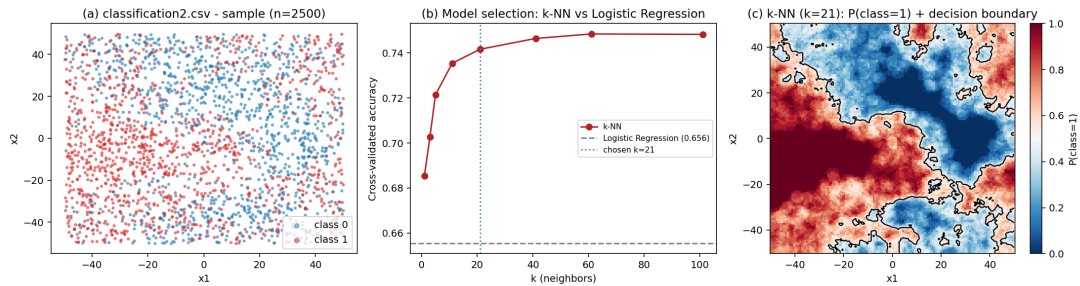


Figure 2.4: (a) Class overlap in `classification2.csv`. (b) Cross-validated accuracy vs k , compared against the logistic regression baseline. (c) Predicted probability of class 1 and decision boundary for $k = 21$.

3. Conclusions

Looking back at all four exercises together, k -NN ended up being the model of choice in three out of four cases, but it needed a completely different configuration every time: $k = 1$ to preserve a sharp discontinuity without smoothing it away, $k = 100$ to filter out heavy noise on a bivariate surface, and $k = 21$ to get a stable boundary through two overlapping classes. There is no single "good" value of k that works across datasets, it really depends on what the data looks like, and cross-validation was the only reliable way I found to figure out which end of that range a given dataset needs.

Exercise 2 was the odd one out, and I think that is worth keeping in mind as much as the other three: when the classes are already well separated, a simple logistic regression did just as well as (and by a small margin, better than) more flexible models, at a fraction of the computational cost. Reaching for k -NN or an SVM there would not have bought anything.

If I had to summarize the main thing I took away from this homework, it is that exploring the data before choosing a model, and then checking that choice with cross-validation instead of trusting intuition, mattered a lot more than knowing the theoretical properties of any single algorithm.